

Introduction/Module overview:

Hello SegFault Survivors stakeholders. We're Group 20 - the Data Analytics team. Our module mainly focuses on data validation, data formatting with timestamp information, logging to the console, and storing simulation information in a file to be utilized during replay. Our initial classes include - ReplayDriver, GlobalClock, Timer, ActionLog, DataLog, and OutputManager.

ReplayDriver class overview:

Hello my name is Apar and I worked on the ReplayDriver class.

This class is responsible for reading the save file line by line. Each line in our save file will correspond with an action or event that an Agent makes. It will then decompose the line to identify what kind of Agent made the move and the Agent id. If the Agent doesn't exist, then the Driver will ask the world to spawn in the respective agent with the given information in the JSON. If the Agent does already exist, then the Driver will ask the world for a reference/pointer to the Agent with the corresponding Agent_id. Then instruct the Agent to perform the task given in the JSON.

GlobalClock & Timer class overview:

Hello my name is Abdulrahman Alanazi and I worked on Both Global Clock and Timer class(es) Global Clock is class that represent the wall clock for the entire simulation. It maintains a single unified time for the whole simulation. Because it's static class, you can't create an instance of it, there is only one global time that exist for the whole program. Global Clock methods include GetTime and Tick and Reset. GetTime returns the current time in ticks. While the Tick method tells the program that the time have passed by specified amount. Reset on the other hand, sets the time to 0.

Moving on to the Timer class. Think of the timer class as a stopwatch, its purpose to have a precise timing measurements. You can create an instance of it with a given name and you can use Start to start the timer and pause and resume the timer if you would like followed by using Stop method to stop the timer. You can also call Reset to reset the member variables and set it back to its default state while preserving the name. You can also call GetTotalTime to get the time in ticks, or you can also call GetTimeInSeconds or GetTimeInMinutes or GetTimeInHours to get the time in more readable units.

ActionLog class overview:

Hello my name is Scott and I worked on the ActionLog class.

ActionLog will be the API that all log-generating classes will need to interface with. ActionLog's primary role is to serve as an interface and validate incoming logs. Log-generating classes will need to call LogEvent() in ActionLog and pass a JSON object. This object must contain four fields: type, id, log_level, and details. Log_level indicates the importance of your log and will be

used by the output manager. This is an integer Enum defined in the output manager so it'll store it as an integer in the JSON object. Type would be the type of object generating this class like Agent or World. It indicates what schema the log is using so we properly instantiate your class during replay. Id is the specific object that this log corresponds to like Agent1. Details will be the main body of your log for things like position or speed. If your JSON object has all of the fields, it will forward the incoming log to DataLog. Finally, it will return a success or failure enum.

DataLog class overview:

Hello my name is Divya and I worked on the DataLog class.

DataLog will serve as the center for data aggregation, data entry formulation, and storage. DataLog receives the validated event information (in the form of a JSON object) from ActionLog. It then calls the static class GlobalClock to get the current global timestamp, adds it to the JSON entry, and stores it in a vector of data entries called mEntries. Apart from creating and storing agent event information, DataLog also has a variety of getter functions that keep track of essential statistics such as the running sum, count, median, and mean based on the field value called "duration" in the JSON object. These are tracked and updated as member variables of the class as and when new data entries are registered in mEntries. There is also a flag member variable named mHasData that tracks if there are any entries in mEntries log, which makes it pretty useful for quick checks instead of iterating through the vector for calculating statistics or for other purposes down the line. DataLog is an internal class for the Data Analytics group. Its functions would be accessible to other groups in the company through ActionLog during live simulation logging and through ReplayDriver during replay of previous simulations.

OutputManager class overview:

Hi, I'm Noyonika, and I worked on the OutputManager class.

This class is a configurable logging system for our simulation framework which routes log messages and finalized simulation data to a JSON file, the console, or both depending on the log levels.

The core of OutputManager is the LogLevel enum. There are four levels: Silent, Normal, Verbose, and Debug. Silent: logs that are only needed for reproduction of the game (ex: movements)

Debug: events for agents (ex: hopped on bus)

Verbose: critical events for agents (ex: initialization, goal finish) and world events (ex: class starting)

Normal: key world events (ex: world initialization, infection started)

For general logging, i wrote `LogMessage()`. You pass in a level and a message string. The method checks whether that message's level meets the current threshold — if it doesn't, it's dropped. If it does, it writes to `std::clog` on the console, and if a file path is set, it also buffers the message as a JSON object with a level and text field into `mBufferedLog`.

Speaking of file output — `SetOutputFile()` sets the path for the JSON log file. It closes any previously open stream first, so you can safely swap files between runs. The actual stream is opened via `ensureOutputStreamOpen()` — only when something actually needs to write. This avoids holding open a file you haven't written to yet.

The most important method is `WriteSimulationOutput()`. This is called by `DataLog` at the end of a simulation run. It pulls all log entries and computed statistics — count, mean, median, min, max — out of the `DataLog` object, serializes them into `mBufferedLog`, and writes the entire thing to the JSON file using `nlohmann::json::dump(2)` for pretty-printing. It also emits a concise human-readable summary line to the console, as long as the level isn't Silent.